



Abdelmalek Essaâdi University
National School of Applied Sciences – Al Hoceima
Department of Data Engineering – 2nd Year

Mini-Project Report

Transformers & RAG Systems

Natural Language Processing

Prepared by:

SAID Salma
TAMZIRT Mohamed

Supervised by:

Pr. Tarik BOUDAA

Academic Year 2024–2025

Contents

1	General Introduction	1
1.1	The NLP Context	1
1.2	The Importance of Transformers	1
1.3	Objective of the Mini-Project	1
1.4	Structure of the Project	1
2	Transformers and Hugging Face	2
2.1	What is a Transformer?	2
2.2	The Role of Pre-Trained Models	2
2.3	Tokenizers	2
2.4	Hugging Face Pipelines	2
2.5	Example of a Pipeline Implementation	2
3	Implemented NLP Tasks	3
3.1	Zero-Shot Text Classification	3
3.2	Sentiment Analysis	3
3.3	Question Answering	3
3.4	Automatic Summarization	3
4	State of the Art on RAG	4
4.1	Definition of RAG	4
4.2	Limits of LLMs Alone	4
4.3	Comparison of Approaches	4
5	Architecture of the RAG Pipeline	6
5.1	Pipeline Steps	6
5.2	Architecture Diagram	6
6	Implementation of the RAG System	8
6.1	The Document Corpus	8
6.2	The Embeddings Model	8
6.3	The Vector Engine: FAISS	8
6.4	The Generative Model	8
6.5	Structure of the Python Project	8
7	Experimentation and Results	9
7.1	Tested Questions	9
7.2	Comparison: Without RAG vs. With RAG	9
7.3	Analysis of Results	9
8	Conclusion, Limits, and Perspectives	11
8.1	Summary of Learnings	11
8.2	Limits of the Current Implementation	11
8.3	Perspectives and Future Work	11

1 General Introduction

1.1 The NLP Context

Natural Language Processing (NLP) is a branch of artificial intelligence that enables computers to understand, interpret, and manipulate human language. Over the past decade, the field has undergone a major paradigm shift. We have moved from rule-based systems and traditional machine learning algorithms (such as SVMs or basic Recurrent Neural Networks) to highly sophisticated deep learning architectures capable of capturing deep semantic nuances and context.

1.2 The Importance of Transformers

The turning point for modern NLP was the introduction of the Transformer architecture by Vaswani et al. in 2017. By replacing sequential processing with the self-attention mechanism, Transformers enabled models to process entire sentences simultaneously. This breakthrough allowed for the training of massive pre-trained language models (such as BERT, GPT, and T5) on vast amounts of text data. These models have achieved state-of-the-art results across virtually all NLP tasks, revolutionizing how we approach machine translation, text generation, and information extraction.

1.3 Objective of the Mini-Project

The primary objective of this mini-project is to bridge the gap between theoretical NLP concepts and practical implementation. By working directly with state-of-the-art models, we aim to understand how these technologies are applied to solve real-world language processing challenges.

1.4 Structure of the Project

To achieve this objective, the project is structured into two main parts:

- **Part 1: NLP Tasks with Hugging Face.** In this part, we explore the Hugging Face `transformers` library. We familiarize ourselves with tokenizers, pre-trained models, and pipelines by implementing four standard NLP tasks: text classification, sentiment analysis, question answering, and automatic summarization.
- **Part 2: Retrieval-Augmented Generation (RAG).** In the second part, we tackle one of the major limitations of current language models: their inability to access external or recent knowledge. We study the RAG architecture and implement a complete system capable of answering questions based on a specific, provided document corpus.

All the implementations and experiments were conducted using a Python Notebook (`transformers_rag_nlp_lab_v2.ipynb`) running in a Google Colab environment with a Tesla T4 GPU.

2 Transformers and Hugging Face

2.1 What is a Transformer?

A Transformer is a deep learning architecture that relies entirely on **attention mechanisms** to draw global dependencies between input and output. Unlike RNNs or LSTMs, which read text sequentially, Transformers use *self-attention* to weigh the importance of all words in a sentence relative to each other at the same time. This parallel processing capability makes them highly efficient to train on large datasets.

2.2 The Role of Pre-Trained Models

Training a Transformer from scratch requires massive amounts of text data and computational power. To solve this, the NLP community adopted the **transfer learning** paradigm. Models are first *pre-trained* on enormous text corpora to learn general language representation (grammar, facts, reasoning abilities). These pre-trained models are then made available to developers, who can either use them directly for "zero-shot" tasks or fine-tune them on small datasets for specific applications.

2.3 Tokenizers

A model cannot understand raw text strings. A **tokenizer** is the crucial first component of any NLP pipeline. It is responsible for splitting the input text into manageable pieces called "tokens" (which can be words, sub-words, or characters) and converting these tokens into numerical identifiers (`input_ids`) that the neural network can process.

2.4 Hugging Face Pipelines

Hugging Face is the leading open-source platform for NLP. It provides the **transformers** Python library, which abstracts the complex underlying PyTorch or TensorFlow code. The simplest way to use these models is through the **pipeline** API, which automatically handles the tokenization, model inference, and output decoding in just a few lines of code.

2.5 Example of a Pipeline Implementation

Below is a brief example of how a pipeline is instantiated and used for a task:

```
1 from transformers import pipeline
2
3 # 1. Initialize the pipeline with a specific task and model
4 sentiment_pipeline = pipeline(
5     task="sentiment-analysis",
6     model="cardiffnlp/twitter-xlm-roberta-base-sentiment"
7 )
8
9 # 2. Provide the input text
10 text = "Ce projet de NLP est vraiment très intéressant !"
11
12 # 3. Get the prediction
13 result = sentiment_pipeline(text)
14 print(result)
15 # Output: [{'label': 'positive', 'score': 0.98}]
```

3 Implemented NLP Tasks

In Part 1 of the project, we implemented four key NLP tasks using Hugging Face pipelines.

3.1 Zero-Shot Text Classification

- **Objective:** Assign a category to a text without having trained the model specifically for those categories.
- **Model Used:** facebook/bart-large-mnli
- **Input:** A text ("*L'équipe a déployé une nouvelle base de données...*") and a list of candidate labels (*informatique, sport, économie, politique*).
- **Output:** The probabilities for each label. The model correctly assigned the highest probability (**0.69**) to the label "*informatique*".

3.2 Sentiment Analysis

- **Objective:** Determine the emotional tone behind a series of words to gain an understanding of the attitudes, opinions, and emotions expressed.
- **Model Used:** cardiffnlp/twitter-xlm-roberta-base-sentiment
- **Input:** A text sentence (e.g., "*Je n'ai pas du tout aimé cette expérience.*").
- **Output:** A sentiment label and a confidence score. The model output "**Negative**" with a score of **0.85**.

3.3 Question Answering

- **Objective:** Extract the exact answer to a user's question from a provided context paragraph (Extractive QA).
- **Model Used:** deepset/xlm-roberta-base-squad2
- **Input:** A **Context** (A text explaining the ENSA Al Hoceima Data Engineering mini-project) and a **Question** ("*Quel est le sujet du mini-projet ?*").
- **Output:** The exact string extracted from the text: "*l'exploration des modèles Transformers*", along with a confidence score.

3.4 Automatic Summarization

- **Objective:** Produce a concise and fluent summary while preserving key information and overall meaning.
- **Model Used:** lincoln/mbart-mlsum-automatic-summarization
- **Input:** A long paragraph detailing the history and mechanism of the Transformer architecture.
- **Output:** A generated short summary: "*Le traitement du langage naturel a franchi un cap important avec l'introduction de l'architecture Transformer en 2017.*"

4 State of the Art on RAG

4.1 Definition of RAG

Retrieval-Augmented Generation (RAG) is a hybrid artificial intelligence framework introduced in 2020. It bridges the gap between traditional information retrieval systems (like search engines) and modern generative Large Language Models (LLMs). When a user asks a question, a RAG system first retrieves relevant documents from an external database, and then feeds those documents to the LLM to generate a highly contextualized and accurate answer.

4.2 Limits of LLMs Alone

While powerful, LLMs operating on their own suffer from several critical flaws:

- **Hallucinations:** When an LLM doesn't know an answer, it often invents highly plausible but entirely false information.
- **Lack of Recent Data:** The knowledge of an LLM is "frozen" at the time its training data was collected. It does not know about events that occurred yesterday.
- **Absence of Private Sources:** An LLM trained on public internet data cannot answer questions about internal enterprise documents, private emails, or proprietary databases.

4.3 Comparison of Approaches

To address these limitations, three main approaches are commonly discussed in the industry. The table below compares them:

Table 1: Comparison: Fine-Tuning, Prompt Engineering, and RAG

Feature	Fine-Tuning	Prompt Engineering	RAG
Principle	Retrain the model's weights using specific, specialized data.	Carefully design the input text to guide the model's behavior.	Retrieve external data dynamically and pass it to the model.
Cost & Effort	High. Requires GPUs, annotated data, and training time.	Low. Requires only experimentation with text.	Medium. Requires setting up a vector database and retrieval logic.
Data Updates	Must completely re-train to add new knowledge.	Limited by the model's maximum context window.	Instant. Simply add new documents to the database.
Traceability	None. Cannot cite sources.	None.	High. Can point exactly to the retrieved document used.

RAG emerges as the most practical solution for knowledge-intensive tasks, as it eliminates hallucinations and provides traceability without the heavy costs of fine-tuning.

5 Architecture of the RAG Pipeline

To build a RAG system, data must flow through a specific sequence of operations. This process is divided into an offline phase (data ingestion) and an online phase (querying).

5.1 Pipeline Steps

1. **Documents:** The raw knowledge base (PDFs, text files, databases) is collected and split into smaller, manageable pieces called "chunks".
2. **Embeddings:** Each chunk is passed through an embedding model, which translates the semantic meaning of the text into a dense mathematical vector.
3. **Vector Indexation:** These vectors are stored in a specialized Vector Database capable of performing rapid similarity searches.
4. **User Query:** The user asks a question in natural language.
5. **Top-K Search:** The user's query is converted into an embedding. The vector database calculates the distance between the query vector and all document vectors, returning the "Top-K" most semantically similar chunks.
6. **Augmented Prompt:** The retrieved chunks are injected into a predefined prompt template alongside the user's original question.
7. **Generated Response:** The generative LLM reads the augmented prompt and crafts a final answer based *strictly* on the provided context.

5.2 Architecture Diagram

The following diagram illustrates the flow of data through the RAG architecture implemented in our project:

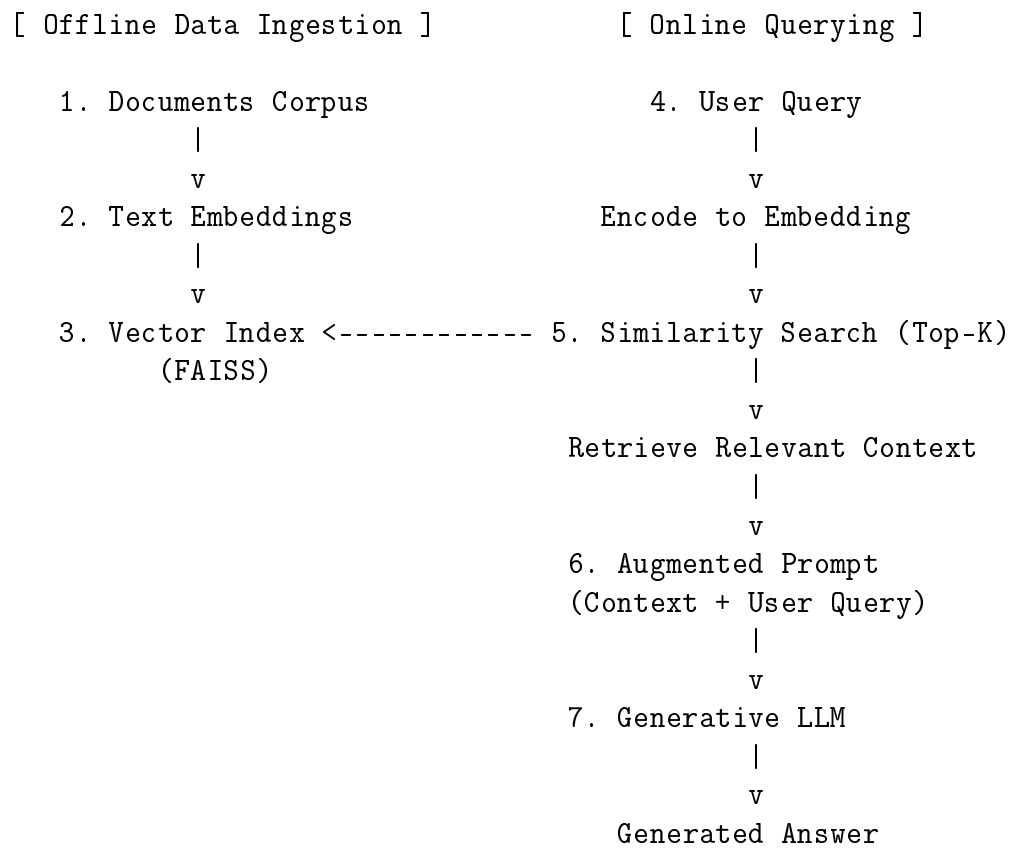


Figure 1: Simplified architecture of the Retrieval-Augmented Generation system.

6 Implementation of the RAG System

We implemented the architecture described above using Python within our Notebook environment. Below are the details of the specific technologies and components utilized.

6.1 The Document Corpus

To ensure the system is self-contained and reproducible, we created a simplified, thematic dataset. It consists of 14 distinct text chunks covering topics highly relevant to our course: Transformer architectures, specific NLP tasks, definitions of RAG, and comparisons with fine-tuning. These documents serve as the external knowledge base.

6.2 The Embeddings Model

To convert our text chunks into vectors, we used the `sentence-transformers` library. Specifically, we selected the model `paraphrase-multilingual-MiniLM-L12-v2`. This is a highly efficient, lightweight model that supports multiple languages (including French) and outputs dense vectors of 384 dimensions.

6.3 The Vector Engine: FAISS

For the indexation and retrieval phase, we utilized **FAISS** (Facebook AI Similarity Search). We implemented an `IndexFlatL2` index, which performs an exact nearest-neighbor search by calculating the Euclidean distance (L2 norm) between the query vector and all the stored document vectors. While exhaustive, it is perfectly suited and highly accurate for our dataset size.

6.4 The Generative Model

For the generation step, we used the Hugging Face `pipeline` for text-to-text generation. The model selected was `google/flan-t5-small`. Flan-T5 is an instruction-tuned model that is particularly adept at following prompts and synthesizing information from provided contexts, making it an excellent lightweight choice for a RAG generator.

6.5 Structure of the Python Project

The implementation is structured sequentially in the `transformers_rag_nlp_lab_v2.ipynb` notebook:

- **Section 2.3.1 - 2.3.3:** Library imports and definition of the document corpus array.
- **Section 2.3.4 - 2.3.5:** Loading the SentenceTransformer model and encoding the corpus into a NumPy array of shape (14, 384).
- **Section 2.3.6 - 2.3.7:** Initializing the FAISS index, adding the vectors, and creating the `rechercher_documents_pertinents()` search function.
- **Section 2.3.8:** Loading the Flan-T5 model and assembling the final `systeme_rag()` function, which orchestrates the retrieval, prompt building, and generation steps.

7 Experimentation and Results

To evaluate the effectiveness of our implementation, we tested the system with several questions related to our corpus. Most importantly, we compared the responses generated by the model **without RAG** (relying solely on its internal, pre-trained weights) versus **with RAG** (using our FAISS retrieval pipeline).

7.1 Tested Questions

We formulated five specific questions:

1. Qu'est-ce qu'un système RAG ?
2. Quelle est la différence entre le fine-tuning et le RAG ?
3. Comment fonctionne le mécanisme d'attention dans les Transformers ?
4. Quels sont les avantages du RAG par rapport à un LLM classique ?
5. Comment les embeddings sont-ils utilisés dans un système RAG ?

7.2 Comparison: Without RAG vs. With RAG

Below is an excerpt of the results for two of the key questions:

Question: Quelle est la différence entre le fine-tuning et le RAG ?

- **Without RAG:** The base model produced a very generic, short, and somewhat confused response, failing to highlight the key differences in cost or data updating.
- **With RAG:** The model generated a precise answer explaining that fine-tuning requires retraining on specific data (costly), whereas RAG allows instant knowledge updates without retraining and provides source traceability.
- **Sources Used:** The system indicated it retrieved Document 13 (Distance: 13.9) and Document 12 (Distance: 14.5).

Question: Comment les embeddings sont-ils utilisés dans un système RAG ?

- **Without RAG:** The model attempted to define embeddings generally but missed their specific operational role within the retrieval pipeline.
- **With RAG:** The model accurately explained that documents are first encoded into embeddings and indexed in a vector base like FAISS. Then, the user's question is encoded to find the most similar document vectors.
- **Sources Used:** The system retrieved Document 9 (Distance: 12.1) and Document 10 (Distance: 13.0).

7.3 Analysis of Results

The experimentation clearly proves the value of the RAG architecture. When operating **without RAG**, the small Flan-T5 model lacked the specific knowledge required to answer technical questions accurately. However, **with RAG**, the exact same model was

able to provide highly precise, factual, and contextualized answers. Furthermore, the RAG system provided transparency: by outputting the retrieved chunks and their FAISS distances, we could directly trace the origin of the model's knowledge, ensuring the absence of hallucinations.

8 Conclusion, Limits, and Perspectives

8.1 Summary of Learnings

This mini-project successfully bridged the gap between theoretical knowledge and practical NLP engineering. In the first phase, we learned how to leverage the Hugging Face ecosystem to rapidly deploy state-of-the-art Transformer models for tasks like classification and summarization.

In the second phase, we successfully architected and built a Retrieval-Augmented Generation (RAG) system from scratch. We gained hands-on experience with vector embeddings via `sentence-transformers`, similarity search using **FAISS**, and context-augmented text generation. The comparative results conclusively demonstrated that combining a retrieval engine with a language model produces a system that is far more accurate, factual, and traceable than an LLM alone.

8.2 Limits of the Current Implementation

While functional, our implementation has several constraints:

- **Small Corpus:** We utilized a simplified dataset of only 14 documents. In a real-world scenario, processing large PDFs involves complex chunking strategies (handling overlaps, tables, and images) that we bypassed.
- **Dependence on Embedding Quality:** The entire system relies on the semantic accuracy of the MiniLM embedding model. If the embedding model fails to place the query and the relevant document close together in the vector space, the generative model will receive the wrong context and fail.
- **Generative Model Capacity:** We used `flan-t5-small` due to compute constraints. While effective with good context, it sometimes struggles with complex reasoning or generating fluent, lengthy French text compared to larger models.

8.3 Perspectives and Future Work

To evolve this project into a production-ready application, several enhancements could be implemented:

- **Robust Document Pipeline:** Integrate tools like LangChain or LlamaIndex to automatically ingest, clean, and chunk massive external knowledge bases (PDFs, Confluence wikis, websites).
- **Advanced Retrieval (Re-ranking):** Implement a two-stage retrieval process. First, use FAISS to retrieve the top 20 documents, then use a Cross-Encoder model to re-rank them, sending only the top 3 most highly relevant chunks to the LLM.
- **Interactive Web Interface:** Wrap the Python logic in a web framework such as Streamlit or Gradio to provide a user-friendly, ChatGPT-like interface where users can upload documents and chat with them in real-time.